arXiv:2512.13845v1 [cs.CE] 15 Dec 2025

Co-simulation errors due to step size changes

Lars T. Kyllingstad*

Abstract

When two simulation units in a continuous-time co-simulation are connected via some variable q, and both simulation units have an internal state which represents the time integral of q, there will generally be a discrepancy between those states due to extrapolation errors. Normally, such extrapolation errors diminish if the macro time step size is reduced. Here we show that, under certain circumstances, step size changes can cause such discrepancies to *increase* even when the change is towards smaller steps.

1 Introduction

Co-simulation is an approach to simulation of coupled dynamical systems where a system of interest is divided into subsystems, each of which is simulated independently of the others, only exchanging information at discrete communication points. Between communication points, each such *simulation unit* is responsible for evolving its own internal state forward in time, for example by integrating a set of ordinary differential equations.

This decoupling of subsystems makes co-simulation especially well suited for multiphysical simulation, distributed computing, and, in an industrial setting, protection of sensitive information about individual subsystems [1]. However, it is also a source of errors. Since simulation units only sample the true values of their inputs at communication points, they have to approximate them during the intervening time intervals. With explicit co-simulation methods, simulation units must extrapolate from past input values, and with implicit methods, they can interpolate input variables between one communication point and the next. Such approximations are by nature inaccurate and can lead to discontinuities at communication points. Thus, co-simulation introduces *coupling errors* on top of errors already present in the numerical computation of subsystem behaviours.

As long as the simulation units are convergent and have continuous behaviour, coupling errors scale as some power of the temporal distance between communication points [2–5], called the *macro time step size*.[1] This means that one can reduce errors by reducing the step size, at the cost of increased computational time. This is used to good effect in variable-step-size co-simulation algorithms, which dynamically adjust the step size during the course of a simulation to maximise performance while keeping errors within certain bounds [5–11].

*SINTEF Ocean, Trondheim, Norway. *lars.kyllingstad@sintef.no*. ORCID 0000-0002-4334-4490.

[1]The prefix “macro” is used to distinguish the time steps between communication points from the internal (micro) time steps used within simulation units. In this paper we will not refer to micro time steps at all, and will therefore often drop the prefix.

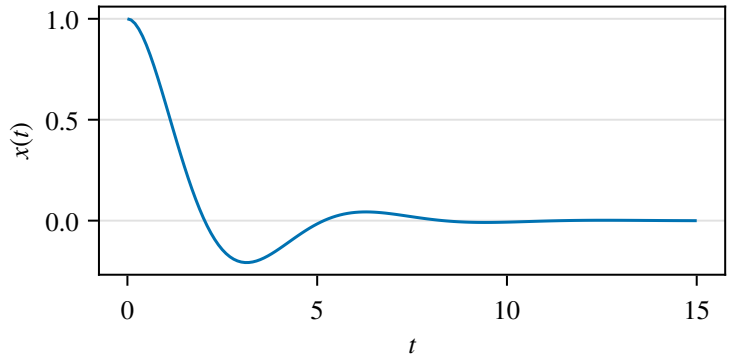


Figure 1: Solution of the damped harmonic oscillator with $m = c = k = 1$ and initial conditions $x(0) = 1$ and $\dot{x}(0) = 0$.

However, as we will show in this paper, in some cases the very act of changing the step size can induce errors, even when the change is towards shorter steps. Specifically, we will show that when a coupling variable is integrated on both sides of a connection, so that the integral on one side is based on an approximation, the resulting coupling errors are proportional to the *difference* between subsequent step sizes.

One can argue that having two instances of the same state in one simulation is redundant and should be avoided. For the sake of numerical accuracy, it would be better to maintain the integrated state in just one simulation unit, and expose it as an output so that other simulation units can access its "true" value. In co-simulation, however, it often makes perfect sense to make simulation units as self-contained as possible, avoiding structural dependencies on other simulation units, to ensure modularity, reusability, and scalability. Presenting a minimal interface against other simulation units is part of this [11].

The described situation typically occurs when a *flow* variable – for example velocity, angular velocity, electrical current, or volumetric flow rate – is exchanged between simulation units and integrated to produce a *displacement* – position, angle, electric charge, or displaced volume, respectively. In this paper, we will present two such examples, in sections 2 and 4, as well as a general derivation of the phenomenon in section 3. We conclude the paper with some observations on how the described errors can be mitigated in section 5.

2 Example 1: Damped harmonic oscillator

As our first example, we consider the case of a damped harmonic oscillator:

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = 0. \tag{1}$$

For the sake of clarity, we shall work in dimensionless units and take the mass, damping coefficient, and spring constant to be unity: $m = c = k = 1$. Figure 1 shows the solution for initial conditions $x(0) = 1$ and $\dot{x}(0) = 0$.

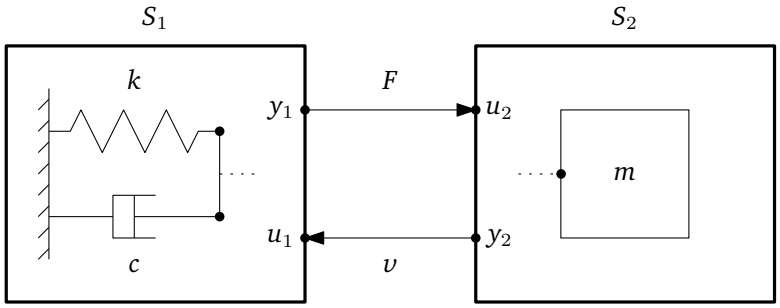


Figure 2: Co-simulation setup of the damped harmonic oscillator.

For the purpose of co-simulation, we envision it in terms of a mass–spring–damper system and split it into two simulation units as illustrated in Figure 2. Simulation unit S_1 represents the spring and the damper. It has one input, u_1, which is the velocity with which the spring and damper are extended; one output, y_1, which is the force by which they oppose the extension; and an internal state, x_1, which is the distance by which they have been extended. The state equation for S_1 is thus

$$\dot{x}_1(t) = \tilde{u}_1(t). \tag{2}$$

The tilde denotes extrapolation, which is necessary since each simulation unit only knows the true value of its input variables at discrete communication points $t[0], t[1], \ldots, t[n]$. Throughout this article, we shall assume the simplest extrapolation scheme, which is just to hold the value constant (i.e., zero-order hold):

$$\tilde{u}_1(t) = u_1[n], \qquad t[n] < t \leq t[n+1]. \tag{3}$$

The angle brackets denote discrete time, that is, $\cdot[n] \equiv \cdot(t[n])$. Consequently, the output equation for S_1 is written

$$y_1[n] = -kx_1[n] - c\tilde{u}_1[n]. \tag{4}$$

Simulation unit S_2 represents the mass. It also has one input, u_2, which is the force acting upon it; one output, y_2, which is its velocity; and two internal states, x_2 and v_2, which are its displacement and velocity, respectively. The state-space equations for S_2 are thus:

$$\dot{x}_2(t) = v_2(t), \tag{5}$$

$$\dot{v}_2(t) = \frac{1}{m}\tilde{u}_2(t), \tag{6}$$

$$y_2[n] = v_2[n]. \tag{7}$$

The coupling constraints, effected at communication points, are:

$$u_1[n] = y_2[n] \qquad \text{(velocity)}, \tag{8}$$

$$u_2[n] = y_1[n] \qquad \text{(force)}. \tag{9}$$

Given that we use zero-order hold, the state equations for the two simulation

Parameter	Symbol	Value
Initial step size	$\Delta t[0]$	0.1
Proportional gain	k_P	0.2
Integral gain	k_I	0.1
Minimum step size	$\Delta t_{\min}$	10^{-5}
Maximum step size	$\Delta t_{\max}$	0.1
Maximum relative step size reduction	$\theta_{\min}$	0.2
Maximum relative step size increase	$\theta_{\max}$	1.2
Absolute tolerance	δ	10^{-6}
Relative tolerance	σ	10^{-6}

Table 1: Parameters for the variable-step-size algorithm and error estimator.

units are easy to solve analytically, and we get:

$$x_1[n+1] = x_1[n] + u_1[n]\Delta t[n], \tag{10}$$

$$x_2[n+1] = x_2[n] + v_2[n]\Delta t[n] + \frac{1}{2m}u_2[n]\Delta t[n]^2, \tag{11}$$

$$v_2[n+1] = v_2[n] + \frac{1}{m}u_2[n]\Delta t[n], \tag{12}$$

where $\Delta t[i] \equiv t[i+1] - t[i]$ are the macro time step sizes.

We then simulate this system using two different co-simulation algorithms: a fixed-step-size algorithm with $\Delta t[i] = \Delta t_{\text{fixed}} = 0.1$ for all i, and a variable-step-size algorithm. In the latter case we use the PI control algorithm described by Kyllingstad, Sadjina and Skjong [11] with an error estimate based on the *Energy-conservation-based co-simulation* (ECCO) method [8]. The parameters used for the PI controller and error estimator are listed in Table 1 for the sake of completeness and reproducibility, but the specific choice of algorithm or error estimator is almost irrelevant here, so we won't dwell further on the details.

Since we have the analytic expressions for the subsystems' behaviour between communication points (10–12), we do not need to perform numerical integration within the simulation units, and any errors we observe next must entirely be artifacts of the co-simulation. We also circumvent any issues that might arise from unfortunate interactions between integration and macro step sizes (see e.g. Skjong and Pedersen [12]).

2.1 The puzzle

Figure 3 shows the results of the two simulations. The first thing we note is that the overall results (left-hand panels) are fairly similar, both to each other and to the accurate solution shown in Figure 1. However, zooming in on the tail (right-hand panels), we see that there is a discrepancy between the displacement states x_1 and x_2 which is especially prominent in the variable-step-size case.

That there is a discrepancy between the simulation units is no surprise in itself. The displacement is obtained through integration of the velocity, and in S_1, the velocity signal from S_2 is sampled and inaccurately reconstructed using zero-order hold. The surprise is that the difference between x_1 and x_2 is significantly

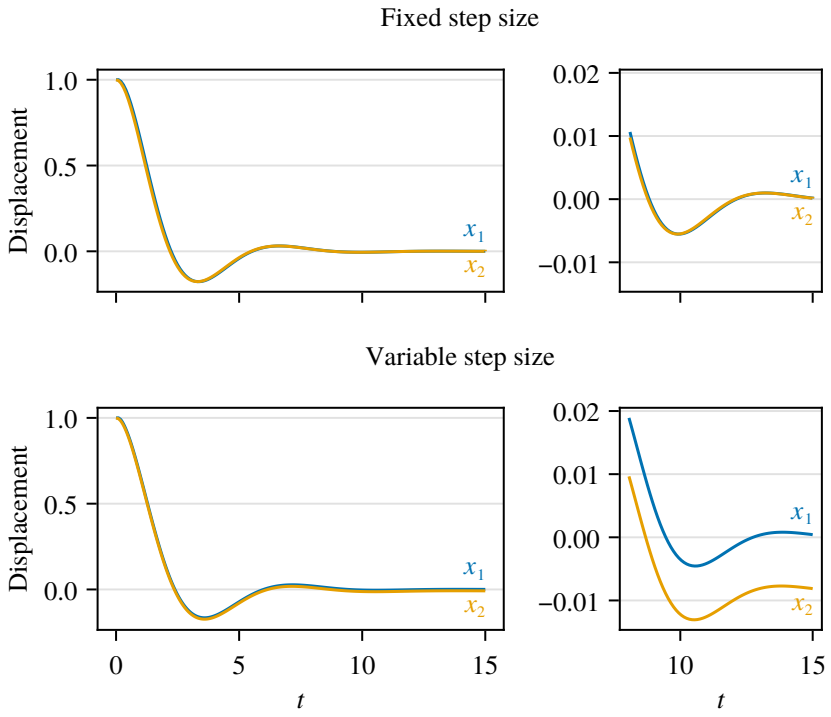


Figure 3: Co-simulated solutions of the harmonic oscillator. The top panels show the solution obtained with a fixed step size; the lower are for the variable-step-size algorithm. The leftmost panels show the entire simulation; the rightmost plots show a zoomed-in picture of the tail ($9 \leq t \leq 15$).

larger in the variable-step-size case, despite the fact that we have configured the controller to use the same or smaller macro step sizes compared to the fixed-step algorithm ($\Delta t_{\text{max}} = \Delta t_{\text{fixed}}$).

To figure out what is happening here, we look more closely at the discrepancy between the displacement states in S_1 and S_2,

$$\Delta x[n] \equiv x_1[n] - x_2[n], \tag{13}$$

alongside a comparison of the step sizes in the two simulations. This is shown in Figure 4. In the fixed-step-size case, we see that the discrepancy initially increases sharply and varies a lot in the first phase of the simulation. This is to be expected, since S_1 lags behind S_2 in terms of its information about the velocity, which also varies a lot in this phase. However, the discrepancy vanishes as the system comes to rest.

In the variable-step-size case, we start off with the same initial step size, and the curves track each other closely in the first couple of time steps. At a point which coincides with the step size algorithm dropping to a smaller Δt, the discrepancy *stops* increasing as sharply and goes into a phase with much smaller oscillations before it too stabilises. However, it ends up stabilising at a larger

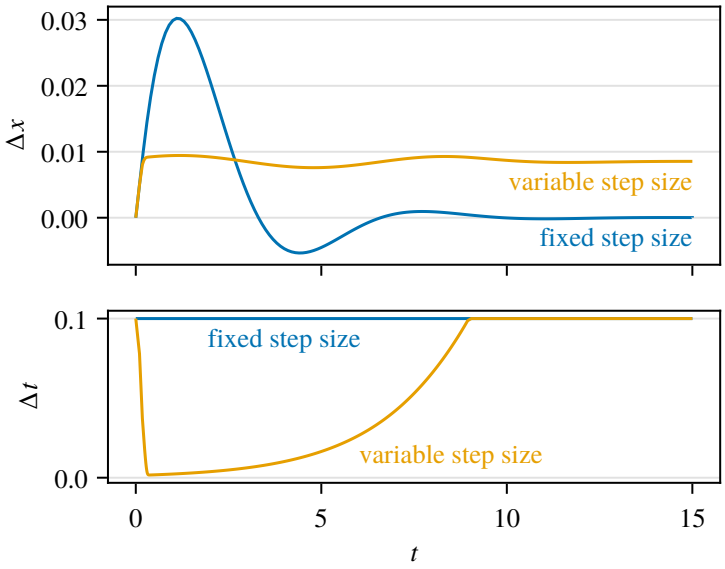


Figure 4: The upper panel shows the discrepancy between the displacement states in S_1 and S_2, $\Delta x = x_1 - x_2$, in the fixed-step-size and variable-step-size cases. The lower panel shows the macro time step size. The simulation was run with an initial velocity $\dot{x}_1 = \dot{x}_2 = 0$.

value than in the fixed-step-size case. The large discrepancy in the early phase appears to have become "frozen in" by the sudden reduction in step size.

Apparently, we are observing a kind of numerical error which is somehow linked to step sizes, though not in the usual way: Rather than being linked to step size magnitude, and diminishing with smaller step sizes, it seems to be linked to step size *changes*. However, it also seems to be heavily dependent on system states, as is shown in Figure 5. Here, we have re-run the simulation with initial velocity $\dot{x}_1[0] = \dot{x}_2[0] = 1$, and now we see that the roles have switched; the variable-step-size algorithm does better in this case.

2.2 The explanation

Note that equations 10–12 for the subsimulators' internal states all take the form of recurrence relations. Applying them repeatedly down to $n = 0$ yields

$$x_1[n] = x_1[0] + \sum_{i=0}^{n-1} u_1[i]\Delta t[i], \tag{14}$$

$$x_2[n] = x_2[0] + \sum_{i=0}^{n-1} \left(v_2[i]\Delta t[i] + \frac{1}{2m} u_2[i]\Delta t[i]^2 \right), \tag{15}$$

$$v_2[n] = v_2[0] + \frac{1}{m} \sum_{i=0}^{n-1} u_2[i]\Delta t[i]. \tag{16}$$

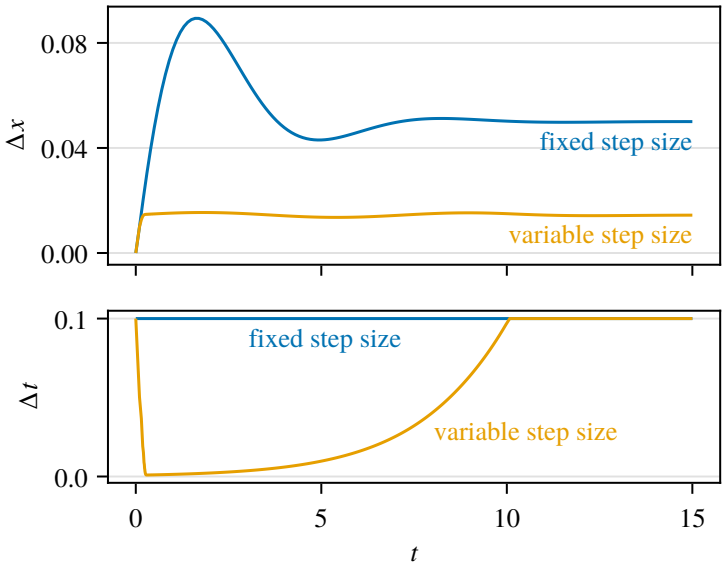


Figure 5: The upper panel shows the discrepancy between the displacement states in S_1 and S_2, $\Delta x = x_1 - x_2$, in the fixed-step-size and variable-step-size cases. The lower panel shows the macro step size. Unlike in Figure 4, the simulation was run with an initial velocity $\dot{x}_1 = \dot{x}_2 = 1$.

If we assume that both displacement states are initialised to the same value ($x_1[0] = x_2[0]$), and make use of the output (7) and constraint (8) equations, which together mean that

$$u_1[i] = y_2[i] = v_2[i], \tag{17}$$

then we find the following compact expression for the state discrepancy (13):

$$\Delta x[n] = -\frac{1}{2m} \sum_{i=0}^{n-1} u_2[i] \Delta t[i]^2. \tag{18}$$

This is an example of the kind of extrapolation errors we discussed in the introduction. The standard ways to deal with it would be:

- Transfer higher-order time derivatives of coupling variables. For this particular system, if $\dot{u}_1[n] = \dot{y}_2[n]$ was added to the coupling constraints, the term of order $\Delta t[i]^2$ would also cancel out entirely.
- Lacking that, perform higher-order input extrapolation within S_1 to obtain *approximations* of $\dot{y}_2$, $\ddot{y}_2$, etc.
- Reduce the step size, since obviously $\Delta x[n] \to 0$ quadratically as $\Delta t[i] \to 0$ for all $i \in \{0, 1, \ldots, n-1\}$.

The last is the simplest, but as we have seen, there is apparently more to that story.

Consider first the fixed-step-size case where $\Delta t[i] = \Delta t_{\text{fixed}} = \text{const}$. Then, we can write

$$\Delta x[n] = -\frac{1}{2m}\Delta t_{\text{fixed}} \sum_{i=0}^{n-1} u_2[i]\Delta t_{\text{fixed}} = -\frac{1}{2}(v_2[n] - v_2[0])\Delta t_{\text{fixed}}, \tag{19}$$

where we made use of the expression (16) for $v_2[n]$ in the last step. This is a dissipative system which eventually comes to rest, so $v_2[n] \to 0$ as $n \to \infty$. We will therefore end up with a constant error of

$$\Delta x[\infty] = \frac{1}{2}v_2[0]\Delta t_{\text{fixed}}. \tag{20}$$

This is exactly what we see in the upper plots in figures 4 and 5.

Then, we turn to the variable-step-size case, and imagine for a moment that we only change the step size once, at communication point K:

$$\Delta t[i] = \begin{cases} \Delta t_1, & i < K \\ \Delta t_2, & i \geq K \end{cases} \tag{21}$$

For $n \geq K$, the displacement discrepancy now becomes

$$\Delta x[n] = -\frac{1}{2m}\Delta t_1 \sum_{i=0}^{K-1} u_2[i]\Delta t_1 - \frac{1}{2m}\Delta t_2 \sum_{i=K}^{n-1} u_2[i]\Delta t_2 \tag{22}$$

$$= -\frac{1}{2}(v_2[K] - v_2[0])\Delta t_1 - \frac{1}{2}(v_2[n] - v_2[K])\Delta t_2 \tag{23}$$

$$= \frac{1}{2}v_2[0]\Delta t_1 + \frac{1}{2}v_2[K](\Delta t_2 - \Delta t_1) - \frac{1}{2}v_2[n]\Delta t_2. \tag{24}$$

The last expression is similar to the fixed-step one (19), except there is now an additional contribution from the step size change which is proportional to the difference between the two step sizes. As the simulation runs its course and the system comes to rest, we get a final discrepancy of

$$\Delta x[\infty] = \frac{1}{2}v_2[0]\Delta t_1 + \frac{1}{2}v_2[K](\Delta t_2 - \Delta t_1). \tag{25}$$

We can also see this in our results, if we consider only the abrupt step size change at the beginning of the simulation and ignore the slow rise in step size thereafter. In Figure 4 the step size drops by an order of magnitude at $t[K] \approx 0.2$, so

$$\Delta t_2 - \Delta t_1 \approx -\Delta t_1 = -0.1. \tag{26}$$

At the same point, the velocity (not plotted) is $v_2[K] \approx -0.2$. Thus, in this case, we get $\Delta x[\infty] \approx 0.1$ which is, to a decent approximation, what we see in the figure.

For the other initial condition, $v_2[0] = 1$, this single-step-size-change simplification works less well, possibly due to a larger value of v_2 during the early macro time steps. In the next section, we will generalise to an arbitrary number of step size changes.

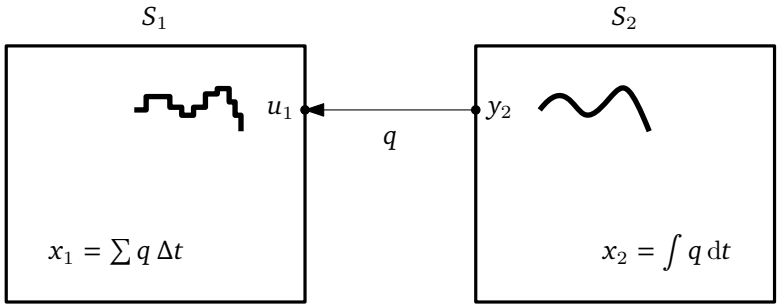


Figure 6: Two systems connected by a flow which accumulates on both sides.

3 General case

Consider now the general case where we have two unspecified systems S_1 and S_2 where S_1 has an input variable u_1 which is connected to S_2's output variable y_2. (Note that while our first example was based on a closed-loop connection, this turns out to be beside the point here.) We shall denote by q the flow that gets transmitted through this connection, so that, at synchronisation point n,

$$u_1[n] = y_2[n] = q[n] = q(t[n]). \tag{27}$$

In both subsystems we have states which represent the integral of $q(t)$, denoted x_1 and x_2, respectively. The setup is illustrated in Figure 6.

In S_2, where $q(t)$ is fully known, the state-space equations are

$$\dot{x}_2(t) = q(t), \tag{28}$$
$$y_2[n] = q[n]. \tag{29}$$

In S_1, where q is represented by the input variable u_1, the corresponding state equation is

$$\dot{x}_1(t) = \tilde{u}_1(t) = u_1[n] = y_2[n] = q[n], \qquad t[n] < t \le t[n+1], \tag{30}$$

where we have again extrapolated the input value using zero-order hold. The value of x_1 at communication point n is then

$$\begin{aligned} x_1[n] &= x_1[n-1] + \int_{t[n-1]}^{t[n]} \mathrm{d}t\ \tilde{u}_1(t) \\ &= x_1[n-1] + q[n-1]\Delta t[n-1] \\ &= x_1[0] + \sum_{i=0}^{n-1} q[i]\Delta t[i]. \end{aligned} \tag{31}$$

The last line is obtained through repeated application of the recurrence relation in the second line.

We will now derive a similarly structured expression for x_2. We assume that the function $q(t)$ is real analytic at $t[n]$, with the Taylor series converging to $q(t)$

for all $t \in [t[n], t[n+1]]$ and all communication points n. In other words, we can write

$$q(t) = \sum_{k=0}^{\infty} \frac{1}{k!} q^{(k)}[n-1](t - t[n])^k, \qquad t[n] \le t \le t[n+1], \tag{32}$$

where $q^{(k)}[i]$ denotes the kth derivative of $q(t)$ at $t = t[i]$. Then, it follows that

$$\begin{aligned} x_2[n] &= x_2[n-1] + \int_{t[n-1]}^{t[n]} \mathrm{d}t\ q(t) \\ &= x_2[n-1] + \sum_{k=0}^{\infty} \frac{1}{k!} q^{(k)}[n-1] \int_{t[n-1]}^{t[n]} \mathrm{d}t\ (t - t[n-1])^k \\ &= x_2[n-1] + \sum_{k=0}^{\infty} \frac{1}{(k+1)!} q^{(k)}[n-1] \Delta t[n-1]^{k+1} \\ &= x_2[0] + \sum_{i=0}^{n-1} \sum_{k=0}^{\infty} \frac{1}{(k+1)!} q^{(k)}[i] \Delta t[i]^{k+1}. \end{aligned} \tag{33}$$

The discrepancy between x_1 and x_2 (at communication points) can now be expressed as

$$\begin{aligned} \Delta x[n] &\equiv x_1[n] - x_2[n] \\ &= x_1[0] - x_2[0] + \sum_{i=0}^{n-1} \left(q[i] \Delta t[i] - \sum_{k=0}^{\infty} \frac{1}{(k+1)!} q^{(k)}[i] \Delta t[i]^{k+1} \right) \\ &= \Delta x[0] - \sum_{i=0}^{n-1} \sum_{k=1}^{\infty} \frac{1}{(k+1)!} q^{(k)}[i] \Delta t[i]^{k+1}. \end{aligned} \tag{34}$$

To see more clearly the link to step size changes, we will make two further assumptions: firstly, that the states x_1 and x_2 are given the same initial value, so that $\Delta x[0] = 0$; and secondly, that all step sizes are so small that $t[n]$ is sufficiently well within the convergence radius of the Taylor series of $q(t)$ at $t[n-1]$ that higher-order terms in $\Delta t[i]$ can be neglected. By the second assumption,

$$q[n] = q[n-1] + q^{(1)}[n-1] \Delta t[n-1] + O(\Delta t[n-1]^2), \tag{35}$$

or

$$q^{(1)}[n-1] = \frac{q[n] - q[n-1]}{\Delta t[n-1]} + O(\Delta t[n-1]). \tag{36}$$

Substituting for this in the $k = 1$ term of Equation 34, the expression for the state discrepancy reduces to

$$\begin{aligned} \Delta x[n] &\simeq -\frac{1}{2} \sum_{i=0}^{n-1} \left(q[i+1] - q[i] \right) \Delta t[i] \\ &= \frac{1}{2} q[0] \Delta t[0] + \frac{1}{2} \sum_{i=1}^{n-1} q[i] (\Delta t[i] - \Delta t[i-1]) - \frac{1}{2} q[n] \Delta t[n-1]. \end{aligned} \tag{37}$$

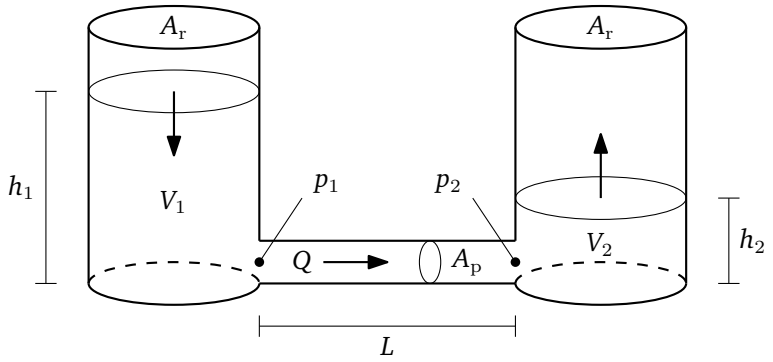


Figure 7: Schematic illustration of the connected fluid reservoirs example.

This shows quite generally that step size changes produce contributions to the state discrepancy which are, at leading order, proportional to both the difference in step size from one step to the next and the flow at the intervening communication point.

4 Example 2: Connected fluid reservoirs

One can argue that the damped harmonic oscillator example we used in section 2 is too pristine and almost seems rigged to produce the exhibited effect. The state x_1, the displacement as seen from the perspective of the spring-damper subsystem, is arguably redundant: It would be natural to treat the displacement of the mass, x_2, as the "ground truth", and it would be easy to add it as an output from S_2 and take it as an input to S_1 instead.

Therefore, we close this paper with another example, illustrated in Figure 7, a model of two fluid reservoirs connected by a pipe. This is a system where it is quite natural to use a flow variable as the interface between two simulation units, since the physical manifestation of it is literally a fluid flow. It is also an example of a system where, although it would be trivial to expose the redundant state as an output of S_2, it simply makes *less sense* for the modeller of the individual subsystems to do so: The integral of the flow is in this case the total amount of fluid which has passed from one reservoir to the other. The natural representation of it inside the simulation units, however, is the total volume of fluid within each reservoir.

We shall employ a highly simplified representation of the system, where the pressure at the pipe orifices is just the hydrostatic pressure at the bottom of the reservoirs,

$$p_i = \rho g h_i = \frac{\rho g V_i}{A_\mathrm{r}}, \tag{38}$$

and the pressure drop from one end of the pipe to the other is given by the Hagen–Poiseuille equation for laminar flow of a viscous, Newtonian fluid:

$$p_1 - p_2 = \frac{8\pi\mu L Q}{A_\mathrm{p}^2}. \tag{39}$$

Physically, this corresponds to a situation where the pipe is very long and narrow relative to the dimensions of the reservoirs, and the fluid flows slowly through it. In these equations, p_1 and p_2 represent pressure at the bottom of each reservoir, V_i is the volume of fluid in each of them, and Q is the volumetric flow rate between them. A_r is the area of the reservoirs' horizontal cross sections (assumed equal), A_p is the cross-sectional area of the pipe, and L is the length of the pipe. ρ and μ are the density and dynamic viscosity of the fluid, respectively, while g is the acceleration of gravity. Since none of the physical details really matter for our purposes, we will, for the sake of clarity, reduce our notation to only two constants, C and R:

$$p_i = \frac{V_i}{C}, \qquad C \equiv \frac{A_\mathrm{r}}{\rho g}, \tag{40}$$

$$p_1 - p_2 = RQ, \qquad R \equiv \frac{8\pi\mu L}{A_\mathrm{p}^2}. \tag{41}$$

(The symbols for these were chosen by analogy with hydraulic capacitance and hydraulic resistance, respectively.)

Again, we split the system into two simulation units S_1 and S_2, where the former represents reservoir 1 and the latter represents reservoir 2 and the pipe. We connect them via a pressure–flow coupling, where the pressure p is the pressure at the interface, i.e., at the bottom of reservoir 1. The setup is shown in Figure 8. The state-space equations for S_1 are

$$\dot{V}_1(t) = -\tilde{u}_1(t), \tag{42}$$

$$y_1[n] = p_1[n] = \frac{V_1[n]}{C}, \tag{43}$$

while for S_2 they are

$$\dot{V}_2(t) = Q(t) = \frac{\tilde{u}_2(t)}{R} - \frac{V_2(t)}{CR}, \tag{44}$$

$$y_2[n] = Q[n]. \tag{45}$$

Note here that y_1 and u_2 represent the interface pressure, p, while u_1 and y_2 represent the interface flow rate, Q. The coupling constraints are thus

$$u_1[n] = y_2[n] \qquad \text{(flow rate)}, \tag{46}$$

$$u_2[n] = y_1[n] \qquad \text{(pressure)}. \tag{47}$$

As in our first example, the state equations (42 and 44) have straightforward analytical solutions, so we will not have to worry about errors introduced by their numerical integration.

We choose parameter values $C = 1$ and $R = 1$ and initial conditions $V_1(0) = 0.6$ and $V_2(0) = 0.4$. We use a "bang-bang" step size controller which is based on the flow rate: If the flow is larger than a certain threshold ($Q_\mathrm{t} = 0.5$), then we use a small step size ($\Delta t_\mathrm{small} = 0.001$), otherwise we use a large step size ($\Delta t_\mathrm{large} = 0.01$). That is,

$$\Delta t[n] = \begin{cases} \Delta t_\mathrm{small}, & y_2[n] > Q_\mathrm{t} \\ \Delta t_\mathrm{small}, & y_2[n] \leq Q_\mathrm{t}. \end{cases} \tag{48}$$

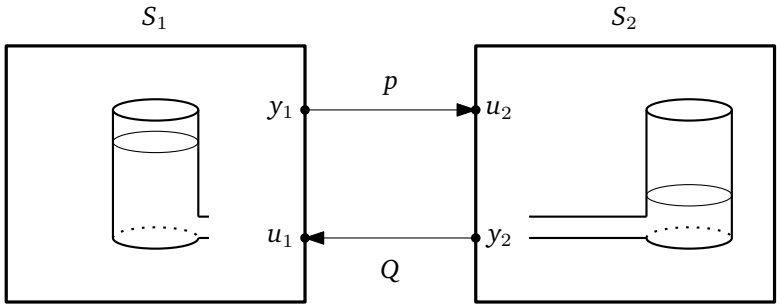


Figure 8: Co-simulation setup of the connected fluid reservoirs example. Note that Q is positive when fluid flows from reservoir 1 to reservoir 2 even though we have chosen causality of the connection to be in the direction from S_2 to S_1. p is the pressure at the interface between the two subsystems, i.e., at the bottom of reservoir 1.

At $t = 1$ we (instantaneously) increase the volume of fluid in S_1 by 1 to increase the flow and precipitate a step size change.

The results are shown in Figure 9. Here, we see that the initial imbalance in fluid level between the two reservoirs decays exponentially, and the fluid volume in both approaches the equilibrium level at 0.5. At $t = 1$ we add a unit of fluid to reservoir 1, and after that, the fluid levels even out at $V_1 \approx V_2 \approx 1$. As in our first example, we see a difference between the fixed- and variable-step-size simulations in the zoomed-in plots on the right. This difference is highlighted in Figure 10, where we again see that the discrepancy between the final states is greater in the variable-step-size case. Here, the discrepancy is calculated as $\Delta V = V - V_1 - V_2$, where V is the total volume of fluid that has been added to the system (i.e., $V = 1$ for $t < 1$ and $V = 2$ for $t \geq 1$).

5 Conclusion

When two simulation units in a co-simulation are connected via some variable q, and both simulation units have an internal state which represents the time integral of q, there will generally be a discrepancy between those states due to extrapolation errors. Such extrapolation errors diminish if the macro time step size is reduced overall, since the reconstruction of q in one simulation unit will converge towards the true functional form of q as defined by the other simulation unit. Conversely, extrapolation errors can be expected to increase with an overall increase in step size.

What we have shown in this paper, however, is that this picture is more nuanced when step sizes are changed in a non-uniform manner over time. A change in step size from Δt to $\Delta t'$ at communication point n produces a leading-order contribution to the accumulated discrepancy between the integral states which is proportional to $q[n](\Delta t' - \Delta t)$.

This is mainly an issue for variable-step-size simulations where q varies over time by orders of magnitude. To understand why this is the case, consider again

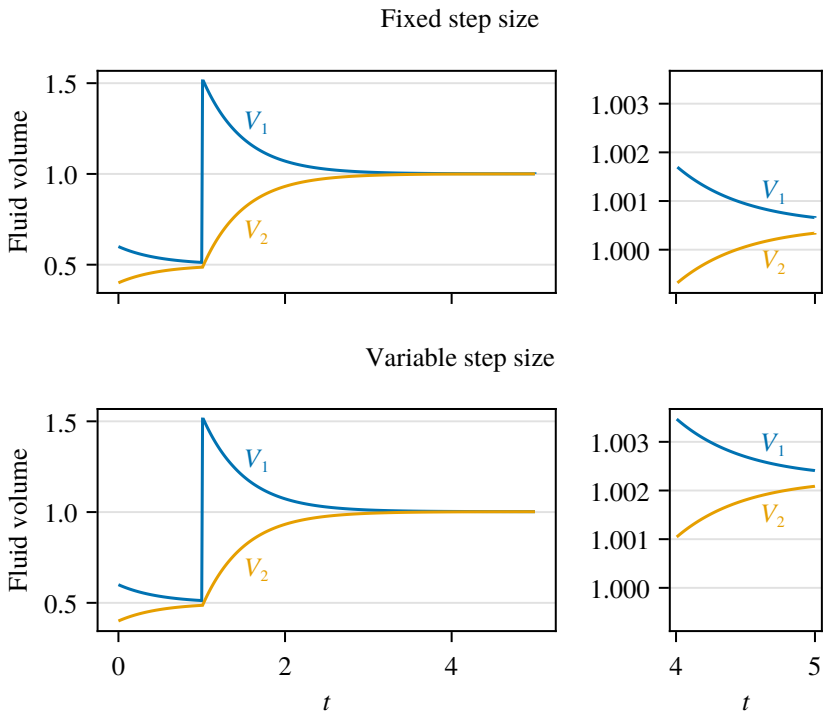


Figure 9: Co-simulation results for the connected fluid reservoirs example. The top panels show the solution obtained with a fixed step size; the lower are for the variable-step-size algorithm. The leftmost panels show the entire simulation; the rightmost plots show a zoomed-in picture of the tail ($4 \leq t \leq 5$).

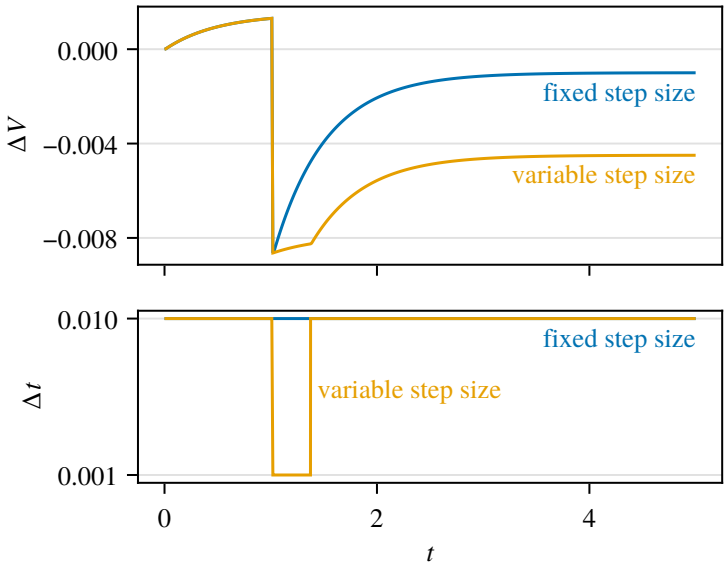


Figure 10: The upper panel shows the discrepancy between the displacement states in S_1 and S_2, $\Delta V = V - V_1 - V_2$, in the fixed-step-size and variable-step-size cases. The lower panel shows the macro time step size.

the general, asymptotic expressions (37) for the state discrepancy:

$$\begin{aligned}\Delta x[n] &\simeq -\frac{1}{2}\sum_{i=0}^{n-1}\left(q[i+1]-q[i]\right)\Delta t[i] \\ &= \frac{1}{2}q[0]\Delta t[0] + \frac{1}{2}\sum_{i=1}^{n-1} q[i]\left(\Delta t[i]-\Delta t[i-1]\right) - \frac{1}{2}q[n]\Delta t[n-1].\end{aligned}$$

In the first line, it is evident that if q does not vary much, so $q[i+1] \approx q[i]$ for all i, the accumulated discrepancy will be small. In the interpretation of the second line, this is because upwards and downwards step size changes will cancel out over time. On the other hand, if q varies a lot, and a large step size change happens to occur when q is large in magnitude, this can incur a discrepancy which does not necessarily get cancelled out later. This is exactly what we observed in our two examples.

To guard against such errors, it may help to limit the aggressiveness of the step size controller in terms of the magnitude of the step size changes it makes. For the PI controller we used in section 2, this amounts to bringing the parameters $\theta_{\min}$ and $\theta_{\max}$ closer to 1, or using lower PI gains.[2] For simulations which have a highly dynamical initial phase, but which eventually relax to a steady state, it can help to start the variable-step-size-controller off at its minimal step size. This is good

[2]In that particular example, the obvious culprit is the parameter $\theta_{\min} = 0.2$, the value of which was just adopted from the original paper [8], which allows the step size to be reduced by as much as 80% in a single jump.

general advice when using a feedback step size controller [11], and especially so in this situation. Finally, one can imagine developing a step size controller which monitors flow-type variables and limits step size changes when they are large. This could be facilitated by a co-simulation interface which supports annotating variables with value ranges and physical units, such as the Functional Mock-up Interface (FMI) [13].

We close by pointing out the main limitations of this study, which are that we have only considered constant input extrapolation (ZOH) and that we used an explicit, Jacobian co-simulation algorithm. Both amount to the "simplest" choice in each respect, the advantage being that we were able to carry out a clear and straightforward analysis of the issue. Since the observed discrepancy is due to extrapolation error, it is to be expected that higher-order extrapolation and the use of implicit co-simulation or Gauss–Seidel iteration would reduce the effect, possibly pushing it to a higher order in Δt. This might be an interesting avenue for future research.

Reproducibility

The code for running the simulations and producing the plots in this paper is available online at `https://doi.org/10.60609/6p8m-0713`. The code is written in the Julia programming language [14] and uses Makie.jl [15] to generate the plots.

Acknowledgements

This work was supported by the Research Council of Norway under the projects *SEACo: Safer, Easier, and more Accurate Co-simulations* (grant number 326710) and *OptiStress: System optimisation and stress testing in co-simulations* (grant number 344238).

The author thanks his colleagues Severin Sadjina and Stian Skjong for interesting discussions and feedback on the work presented herein.

References

[1] Cláudio Gomes et al. 'Co-simulation: A Survey'. In: *ACM Computing Surveys* 51.3, 49 (May 2018), pp. 1–33. DOI: 10.1145/3179993.

[2] R. Kübler and W. Schiehlen. 'Two Methods of Simulator Coupling'. In: *Mathematical and Computer Modelling of Dynamical Systems* 6.2 (2000), pp. 93–113. DOI: 10.1076/1387-3954(200006)6:2;1-M;FT093.

[3] Martin Arnold and Michael Günther. 'Preconditioned Dynamic Iteration for Coupled Differential-Algebraic Systems'. In: *BIT Numerical Mathematics* 41.1 (Jan. 2001), pp. 1–25. DOI: 10.1023/A:1021909032551.

[4] Martin Arnold, Christoph Clauss and Tom Schierz. 'Error analysis and error estimates for co-simulation in FMI for model exchange and co-simulation V2.0'. In: *Archive of Mechanical Engineering* 60.1 (2013), pp. 75–94. DOI: 10.2478/meceng-2013-0005.

[5] Slaven Glumac and Zdenko Kovačić. 'Defect Analysis of a Non-Iterative Co-Simulation'. In: *Mathematics* 11.6, 1342 (2023). DOI: 10.3390/math11061342.

[6] Martin Busch and Bernhard Schweizer. 'An explicit approach for controlling the macro-step size of co-simulation methods'. In: *Proceedings of the 7th European Nonlinear Dynamics Conference*. ENOC 2011 (Rome, Italy, 24th–29th July 2011). Ed. by Davide Bernardini, Giuseppe Rega and Francesco Romeo. 2011. ISBN: 9788890623424.

[7] Tom Schierz, Martin Arnold and Christoph Clauß. 'Co-simulation with communication step size control in an FMI compatible master algorithm'. In: *Proceedings of the 9th International MODELICA Conference* (Munich, Germany, 3rd–5th Sept. 2012). Ed. by Martin Otter and Dirk Zimmer. Linköping Electronic Conference Proceedings 76. 19th Nov. 2012, pp. 205–214. ISBN: 978-91-7519-826-2. DOI: 10.3384/ecp12076205.

[8] Severin Sadjina et al. 'Energy conservation and power bonds in co-simulations: non-iterative adaptive step size control and error estimation'. In: *Engineering with Computers* 33 (2017), pp. 607–620. DOI: 10.1007/s00366-016-0492-8.

[9] T. Meyer, J. Kraft and B. Schweizer. 'Co-Simulation: Error Estimation and Macro-Step Size Control'. In: *Journal of Computational and Nonlinear Dynamics* 16.4, 041002 (Apr. 2021). DOI: 10.1115/1.4048944.

[10] Yohan Éguillon, Bruno Lacabanne and Damien Tromeur-Dervout. 'F_3ORNITS: a flexible variable step size non-iterative co-simulation method handling subsystems with hybrid advanced capabilities'. In: *Engineering with Computers* 38 (Aug. 2022), pp. 4501–4543. DOI: 10.1007/s00366-022-01610-z.

[11] Lars T. Kyllingstad, Severin Sadjina and Stian Skjong. 'Error estimation and step size control with minimal subsystem interfaces'. In: *Simulation Modelling Practice and Theory* 145, 103209 (Dec. 2025). DOI: 10.1016/j.simpat.2025.103209.

[12] Stian Skjong and Eilif Pedersen. 'On the numerical stability in dynamical distributed simulations'. In: *Mathematics and Computers in Simulation* 163 (Sept. 2019), pp. 183–203. DOI: 10.1016/j.matcom.2019.02.018.

[13] The Modelica Association. *Functional Mock-up Interface specification*. Version 3.0.2. 27th Nov. 2024. URL: https://fmi-standard.org (visited on 11/12/2025).

[14] Jeff Bezanson et al. 'Julia: A Fresh Approach to Numerical Computing'. In: *SIAM Review* 59.1 (2017), pp. 65–98. DOI: 10.1137/141000671.

[15] Simon Danisch and Julius Krumbiegel. 'Makie.jl: Flexible high-performance data visualization for Julia'. In: *Journal of Open Source Software* 6.65, 3349 (1st Sept. 2021). DOI: 10.21105/joss.03349.